X2 Capstone 2: a unified Fabric solution

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track X: Capstones **2**
X2 Capstone 2: a unified Fabric solution 3
X2-L1 Capstone 2: a unified Fabric solution 3
Track X best-practice checklist 8
Appendix: solutions and answer keys **9**

Track X: Capstones

Three projects that combine the tracks, each checked against tests or a hidden key.

X2 Capstone 2: a unified Fabric solution

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Mirror Databricks gold into OneLake, build a Direct Lake model and verify Copilot output.

X2-L1 Capstone 2: a unified Fabric solution

Concept

This capstone brings Track C together, with Track A gold tables as the source. You build one reporting path in Fabric and log every Copilot output you use.

The brief. Regional analysts need a weekly reach report on the NAT tracker. The gold table `nat_metrics` lives in Unity Catalog on Azure Databricks. Build:

1. A **mirrored Azure Databricks catalog** item for the gold schema. No data moves: Fabric mirrors the catalog structure and reads the Delta files through shortcuts (C3-L3).
2. A **Direct Lake on OneLake** semantic model over the mirrored tables, with a star schema and `DIVIDE` in every ratio measure (C6).
3. A **Power BI report** with a wave slicer and a region slicer, drafted with Copilot.
4. **Access** rebuilt in Fabric. Unity Catalog grants and row filters do not carry over, so analysts are Viewers and OneLake security roles limit rows where needed (C9-L2).
5. A **verification log**: one row per Copilot output you kept, with how you checked it (C8-L3).

Constraints to design around (Microsoft Learn, checked September 2026):

- Tables with row filters or column masks, views and materialized views do not mirror. Publish a gold table that is safe to share.
- One connection credential reads every mirrored table.
- Direct Lake on OneLake never falls back to DirectQuery. A table over the SKU guardrail fails to frame, so compact gold with `OPTIMIZE`.
- Copilot in Power BI is GA. Copilot output can be inaccurate, and a person able to judge it reviews it.

What you deliver: the workspace items, your verification log (template in the lab sheet), and a one-page design note answering the design exercise

below. You need a Fabric capacity with Copilot enabled for step 3. Without one, do steps 1, 2, 4 and 5 and draft the measures by hand.

Why it matters

Most real reporting work crosses platforms: data engineered in one place, modelled and consumed in another. Each boundary drops something: permissions, freshness guarantees, a filter. This capstone makes you find each boundary on purpose, decide what to rebuild on the far side, and prove that the numbers in the report match the numbers in gold.

Worked example: weak code

```
❶ Mirror: gold schema including v_regional_reach (a view) and nat_metrics
    (row filter on region)
  2 Model: Direct Lake on SQL endpoint, behaviour Automatic
❷ Measures: accepted from Copilot as generated
❸ Access: "Unity Catalog already restricts regions"
```

1. **Line 1.** Neither object mirrors: views and tables with row filters are unsupported.
2. **Line 3.** No check against gold. Copilot output can be inaccurate.
3. **Line 4.** Unity Catalog permissions do not carry over to Fabric.

Worked example: strong code

```
❶ Mirror: gold schema with nat_metrics_shareable (no row filter), auto-sync
    on
  2 Model: Direct Lake on OneLake, star schema, DIVIDE in every ratio
❷ Measures: Copilot drafts, each checked against a SQL total from gold,
    logged
❸ Access: analysts as Viewers, OneLake security role per region on the fact
    table
```

1. **Line 1.** A gold table designed to be mirrored.
2. **Line 3.** Every kept output has a check and a log row.
3. **Line 4.** Row rules rebuilt on the Fabric side.

Decide what each boundary drops, rebuild it on the far side, and prove the report matches gold.

Recap card

- **Capstone 2 path** Shareable gold table, mirrored catalog, Direct Lake on OneLake model, Fabric access, Copilot report, verification log.
- **What does not cross the mirror** Views, tables with row filters or masks, and Unity Catalog permissions.

- **A verification log row** Output id, prompt, check method (known data, second method, read-through), result. Failed checks are not published.

Practice

Exercise 1. Multiple choice (level 1)

Your gold schema holds `nat_metrics` (a Delta table with no policies), `v_regional_reach` (a view) and `nat_metrics_secure` (a row filter on region). Which appear in the mirrored catalog?

- a) nat_metrics only
- b) All three
- c) nat_metrics and v_regional_reach
- d) nat_metrics and nat_metrics_secure, with the row filter applied

Why?

- r1) Views and tables with row filters or column masks are not supported for Unity Catalog mirroring.
- r2) Only tables whose names start with nat_ are mirrored.

Exercise 2. Put in order (level 2)

Order the build.

- Draft the report with Copilot and log each checked output
- Set Viewer roles and OneLake security roles
- Publish nat_metrics_shareable in gold and run OPTIMIZE
- Build the Direct Lake on OneLake semantic model
- Create the mirrored Azure Databricks catalog item

Exercise 3. Fill the gap (level 2)

Complete the reach share measure the report uses, so a zero base shows an empty cell and the wave and region slicers still apply.

```
1 Region share =
2 VAR this_region = SUM ( fact_reach[weighted_reach] )
3 VAR all_regions = CALCULATE ( SUM ( fact_reach[weighted_reach] ), ALL
    ( _____(1) ) )
4 RETURN _____(2) ( this_region, all_regions )
```

Exercise 4. Write the function (level 3)

Write `log_problems(entries: list[dict[str, str]]) -> list[str]` that checks the Copilot verification log. Each entry needs non-empty `output_id`, `prompt`, `check_method` and `result`. `check_method` must be one of `"known_data"`,

`"second_method"` or `"read_through"`. `result` must be `"pass"` or `"fail"`. Return one message per problem, formatted `"<output_id>: <problem>"` with problems `missing <field>`, `unknown check_method` or `unknown result`, in entry order. Use `"?"` as the id when `output_id` is missing or empty. An entry with result `"fail"` also adds `"<output_id>: failed check, do not publish"`.

Your code must pass these checks:

- A complete, passing entry has no problems.
- Report missing fields and unknown values in entry order, with '?' for a missing id.
- A failed check is flagged so it is not published.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
REQUIRED_FIELDS = ...

# Module constant.
CHECK_METHODS = ...

# Module constant.
RESULTS = ...

# Problems in a Copilot verification log.
def log_problems(entries: list[dict[str, str]]) -> list[str]: ...
```

Starter code:

```
def log_problems(entries):
    return []
```

Exercise 5. Architecture design (level 4)

Write your design note's key decisions.

Options: Direct Lake on OneLake, with OPTIMIZE keeping gold under the file guardrail, Import mode with a nightly refresh.

1. Semantic model storage mode
2. Limiting analysts to their region
3. Accepting a Copilot-drafted measure

Practical: Capstone 2: build and verify the NAT reach report

Platform: Fabric trial workspace. Run this in a Fabric trial workspace connected to an Azure Databricks workspace with Unity Catalog. Copilot needs a paid F2 or higher capacity (lesson C8-L1). Keep your verification log

as a table with columns: output_id, prompt, output summary, check_method, check detail, result, checked_by, date.

1. In Databricks, create gold table `nat_metrics_shareable` with no row filter or column mask, holding only data every analyst may see. Run OPTIMIZE on it.
2. In Fabric, create a mirrored Azure Databricks catalog item for the gold schema. Confirm that nat_metrics_shareable appears and that any view or filtered table does not.
3. Create a semantic model in Direct Lake on OneLake mode over the mirrored tables. Build a star schema with dim_region and dim_wave.
4. Add measures for weighted reach and region share with DIVIDE. For each measure Copilot drafts, compare the value for one wave with a SQL total from gold, and log the result.
5. Give analysts the Viewer role. Create a OneLake security role per region that filters the fact table, and test it with a test account.
6. Draft the report pages with Copilot. Log each visual you keep, with the check you used.
7. Run log_problems (exercise 4) on your log. Fix every problem before sharing the report.
8. Write a one-page design note covering storage mode, row security and verification, and the decisions from exercise 5.

Sources

- Unity Catalog mirroring: metadata only via shortcuts; views, materialized views, streaming and tables with row filters or column masks are unsupported; UC permissions do not carry over; one connection credential. (verified: https://learn.microsoft.com/en-us/fabric/mirroring/azure-databricks (see lesson C3-L3))
- Direct Lake on OneLake never falls back to DirectQuery; guardrails by SKU. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/direct-lake-overview (see lesson C6-L3))
- Copilot in Power BI is GA; Copilot requires paid F2+ or P1+; output can be inaccurate and needs review. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/copilot-ai-feature-state and copilot-privacy-security (see lessons C8-L1 and C8-L3))

Track X best-practice checklist

☐ Read the brief and restate the goal before you touch code.

☐ Work to the CLAUDE.md standard, not to the starting code.

☐ Run the acceptance tests before and after each change.

☐ Record every Copilot or assistant output you rely on, with how you checked it.

☐ Compare your work with the reference or key only after you submit.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

X2-L1 Capstone 2: a unified Fabric solution

X2-L1 exercise 1

Hints. (1) Lesson C3-L3. (2) Which object types are unsupported? (3) Views and filtered tables.

Answer. a) nat_metrics only Because: r1) Views and tables with row filters or column masks are not supported for Unity Catalog mirroring.

Plan a gold table that is safe to mirror, and rebuild row rules in Fabric.

Common wrong answers.

- `b`: Views and tables with row filters do not mirror.
- `c`: Views are not supported.
- `d`: Tables with row filters do not mirror, and Unity Catalog policies are not carried over.
- `a|r2`: Right answer, wrong reason. Names do not matter. Object type and policies do.

X2-L1 exercise 2

Hints. (1) What must exist before mirroring? (2) Model before report. (3) Access before sharing.

Answer.

1. Publish nat_metrics_shareable in gold and run OPTIMIZE
2. Create the mirrored Azure Databricks catalog item
3. Build the Direct Lake on OneLake semantic model
4. Set Viewer roles and OneLake security roles
5. Draft the report with Copilot and log each checked output

Source, mirror, model, access, then the report.

Common wrong answers.

- Mirror what is ready: publish and compact the shareable table first.
- Set access before analysts see the report. A report shared early leaks every row.

X2-L1 exercise 3

Hints. (1) Which filter should the denominator ignore? (2) Region only. (3) `dim_region` and `DIVIDE`.

Answer.

```
Region share =
VAR this_region = SUM ( fact_reach[weighted_reach] )
VAR all_regions = CALCULATE ( SUM ( fact_reach[weighted_reach] ), ALL
    ( dim_region ) )
RETURN DIVIDE ( this_region, all_regions )
```

The denominator keeps the wave filter and drops only the region filter. DIVIDE returns BLANK on a zero base.

Common wrong answers.

- g1 = `fact_reach`: ALL (fact_reach) removes the wave slicer too. Remove only the region filter.

X2-L1 exercise 4

Hints. (1) Keep allowed values in constants. (2) Missing and empty both count as missing: `not entry.get(name)`. (3) Check fields in order, then check_method, then result.

Answer.

```python
REQUIRED_FIELDS = ("output_id", "prompt", "check_method", "result")
CHECK_METHODS = frozenset({"known_data", "second_method", "read_through"})
RESULTS = frozenset({"pass", "fail"})


def log_problems(entries: list[dict[str, str]]) -> list[str]:
    """Problems in a Copilot verification log.

    Args:
        entries: One dict per Copilot output kept in the solution.

    Returns:
        Messages such as "M2: missing prompt", in entry order.
    """
    problems: list[str] = []
    for entry in entries:
        output_id = entry.get("output_id") or "?"
        for name in REQUIRED_FIELDS:
            if not entry.get(name):
                problems.append(f"{output_id}: missing {name}")
        method = entry.get("check_method")
        if method and method not in CHECK_METHODS:
            problems.append(f"{output_id}: unknown check_method")
        result = entry.get("result")
        if result and result not in RESULTS:
```

```
            problems.append(f"{output_id}: unknown result")
        if result == "fail":
            problems.append(f"{output_id}: failed check, do not publish")
    return problems
```

"Looked fine" is not a check method. A log that allows it records opinions, not verification.

X2-L1 exercise 5

Hints. (1) Which mode reads mirrored Delta without a copy? (2) Where do row rules live after mirroring? (3) What makes a Copilot output verified?

Answer.

- Direct Lake on OneLake reads the mirrored Delta files without a copy and never falls back. Compaction keeps it within guardrails.
- Unity Catalog policies do not carry over, and filtered tables do not mirror. Rebuild row rules with OneLake security.
- Every kept Copilot output is checked against an independent number and logged.

Direct Lake on OneLake, OneLake security for rows, and a logged check for every Copilot output.

Common wrong answers.

- mode = `import`: Import copies the data and adds a refresh to fail. Direct Lake reads the mirrored files directly.
- rows = `uc_filter`: Tables with row filters do not mirror, and Unity Catalog permissions do not apply in Fabric.
- verify = `looks`: Plausible is not verified. Compare with a number from gold.